

ЛАБОРАТОРНА РОБОТА №1.

РОЗПОДІЛЕНА ОБРОБКА ДАНИХ В APACHE HADOOP ТА APACHE HIVE

Мета роботи – ознайомитися з архітектурою та принципами роботи розподіленої файлової системи HDFS; набути практичних навичок роботи з командним рядком Hadoop; вивчити основи створення сховищ даних в Apache Hive (DDL, DML операції); опанувати методи оптимізації зберігання даних (Partitioning, Bucketing) та реалізувати MapReduce задачу мовою Python.

ОСНОВНІ ТЕОРЕТИЧНІ ВІДОМОСТІ

Екосистема Apache Hadoop

Apache Hadoop – це фреймворк для зберігання та обробки великих масивів даних у розподіленому середовищі. Його ядром є два компоненти:

- Hadoop Common - бібліотеки та утиліти, необхідні для роботи інших модулів.
- HDFS (Hadoop Distributed File System) - файлова система, призначена для зберігання великих файлів (терабайти, петабайти) з забезпеченням відмовостійкості через реплікацію блоків даних (за замовчуванням – 3 репліки). Архітектуру HDFS зображено на рисунку 1.1.
- YARN (Yet Another Resource Negotiator): система управління ресурсами кластера та планування завдань.
- Hadoop MapReduce - система для паралельної обробки великих наборів даних, що базується на YARN.

HDFS Architecture

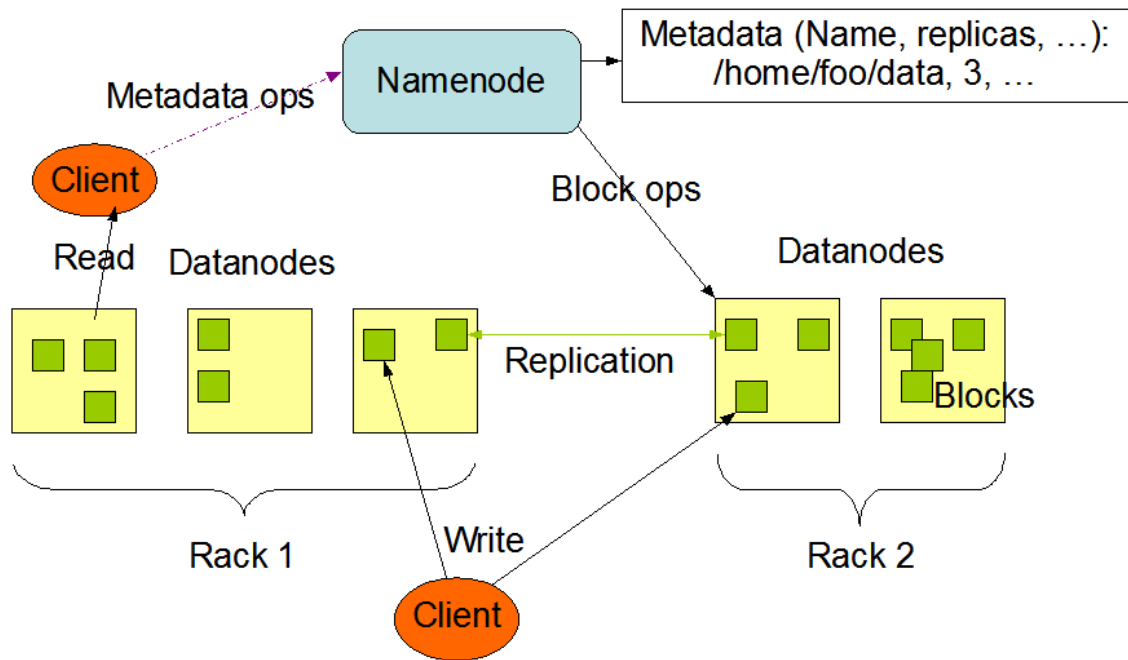


Рисунок 1.1 - Архітектура HDFS

Основні характеристики архітектури HDFS:

1. Блокова структура. HDFS розбиває великі файли на менші, безперервні частини, які називаються блоками або "чанками". Типовий розмір блоку становить 64 МБ або 128 МБ.
2. Ролі вузлів кластера:
 - NameNode: Керує простором імен файлової системи та відповідає за розбиття файлів на блоки.
 - DataNodes: Вузли, на яких фізично зберігаються блоки даних, розподілені по всьому кластеру.
3. Відмовостійкість. Для захисту від втрати даних кожен блок реплікується (копіюється) на кілька вузлів. Стандартний коефіцієнт реплікації становить 3, що дозволяє системі прозоро відновлювати доступ до даних у разі збою окремого вузла.
4. Локальність даних. Архітектура HDFS дозволяє обчислювальним рушіям (наприклад, Spark або MapReduce) обробляти дані безпосередньо на тих вузлах, де вони зберігаються (data locality), що мінімізує навантаження на мережу.

Концепція MapReduce

Концепція MapReduce – це модель програмування для паралельної обробки великих обсягів даних, яка дозволяє розбивати великі задачі з обробки даних на менші, незалежні завдання. Ці завдання виконуються паралельно на кластері серверів, а їхні результати потім об'єднуються для отримання фінального результату. Процес обробки даних у MapReduce зазвичай проходить наступні стадії:

1. Input. Вхідні файли розбиваються на частини (splits).
2. Map:
 - Кожен блок даних обробляється окремим процесом (Mapper).
 - Функція Map перетворює вхідні елементи з однієї форми в іншу. Вона приймає пари «ключ-значення» і генерує нуль або більше проміжних пар «ключ-значення». Наприклад, у задачі підрахунку слів (Word Count) Mapper розбиває текст на слова та для кожного слова генерує пару (слово, 1).
 - Shuffle and Sort (Перемішування та сортування). Це етап, який відбувається між Map та Reduce. Hadoop автоматично сортує проміжні пари за ключем. Всі пари з однаковим ключем передаються (shuffled) на один і той самий Reducer. Це гарантує, що всі дані, необхідні для обробки конкретного ключа, опиняться в одному місці.
3. Reduce:
 - Reducer отримує ключ і колекцію всіх значень, пов'язаних із цим ключем.
 - Функція Reduce агрегує ці значення (наприклад, сумує числа, обчислює середнє або фільтрує дані) і генерує фінальний результат. *Приклад:* У задачі Word Count Reducer отримує (слово, [1, 1, 1...]) і підсумовує одиниці, видаючи фінальну кількість входжень слова (рисунки 1.1 та 1.2).
4. Output. Вихідні дані з результатом записується у розподілену файлову систему (наприклад, HDFS).

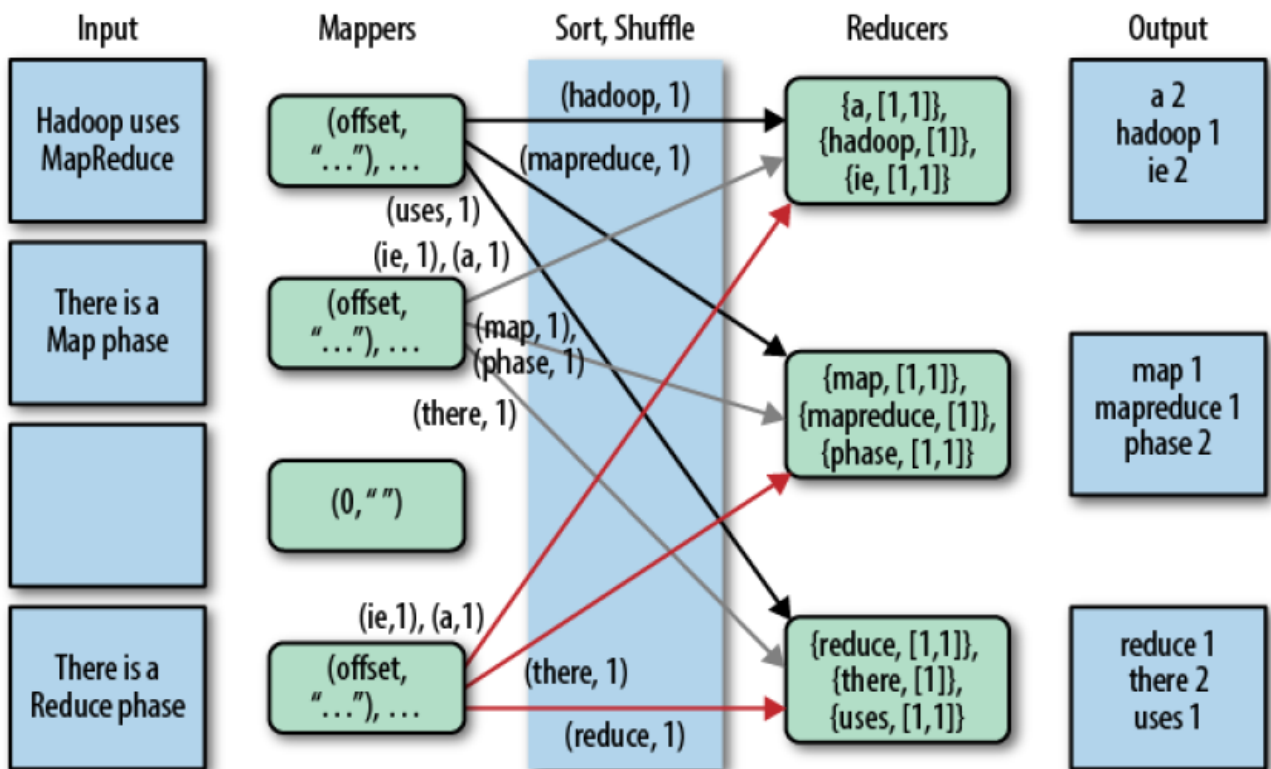


Рисунок 1.2 - Підрахунок слів на основі MapReduce

Apache Hive (далі Hive) – це інструмент сховища даних, що надає абстракцію таблиць поверх даних, розміщених у розподіленій файловій системі (наприклад, HDFS), інших сховищах (HBase) тощо. Основна мета Hive — дозволити користувачам, знайомим з SQL, легко працювати з даними в екосистемі Hadoop, не вивчаючи складні низькорівневі API, такі як Java MapReduce.

Архітектура Apache Hive [3] складається з таких елементів (рисунок 1.3):

1. Рівень клієнтських інтерфейсів (Hive Interfaces). Різні способи, якими користувачі або додатки можуть взаємодіяти з системою:
 - CLI (Command-Line Interface): Інтерфейс командного рядка для безпосереднього введення запитів.
 - HWI (Hive Web Interface): Веб-інтерфейс для роботи з Hive.
 - Thrift Server, JDBC та ODBC: Компоненти, що забезпечують програмний доступ та дозволяють підключати зовнішні інструменти (наприклад, BI-системи) до Hive.
2. Ядро Hive (Hive Core) містить такі модулі:

- Driver. Центральний модуль, куди надходять усі команди та запити. Драйвер відповідає за компіляцію, оптимізацію та виконання запитів. Драйвер перетворює SQL-подібні запити на серію завдань (зазвичай MapReduce), які виконуються на кластері.
- Metastore (Метастор). Окрема база даних (зображена як циліндр), де зберігаються метадані про структуру таблиць, типи колонок та розташування даних у файловій системі.

3. Рівень Hadoop. Ніве побудовано поверх компонентів Hadoop.

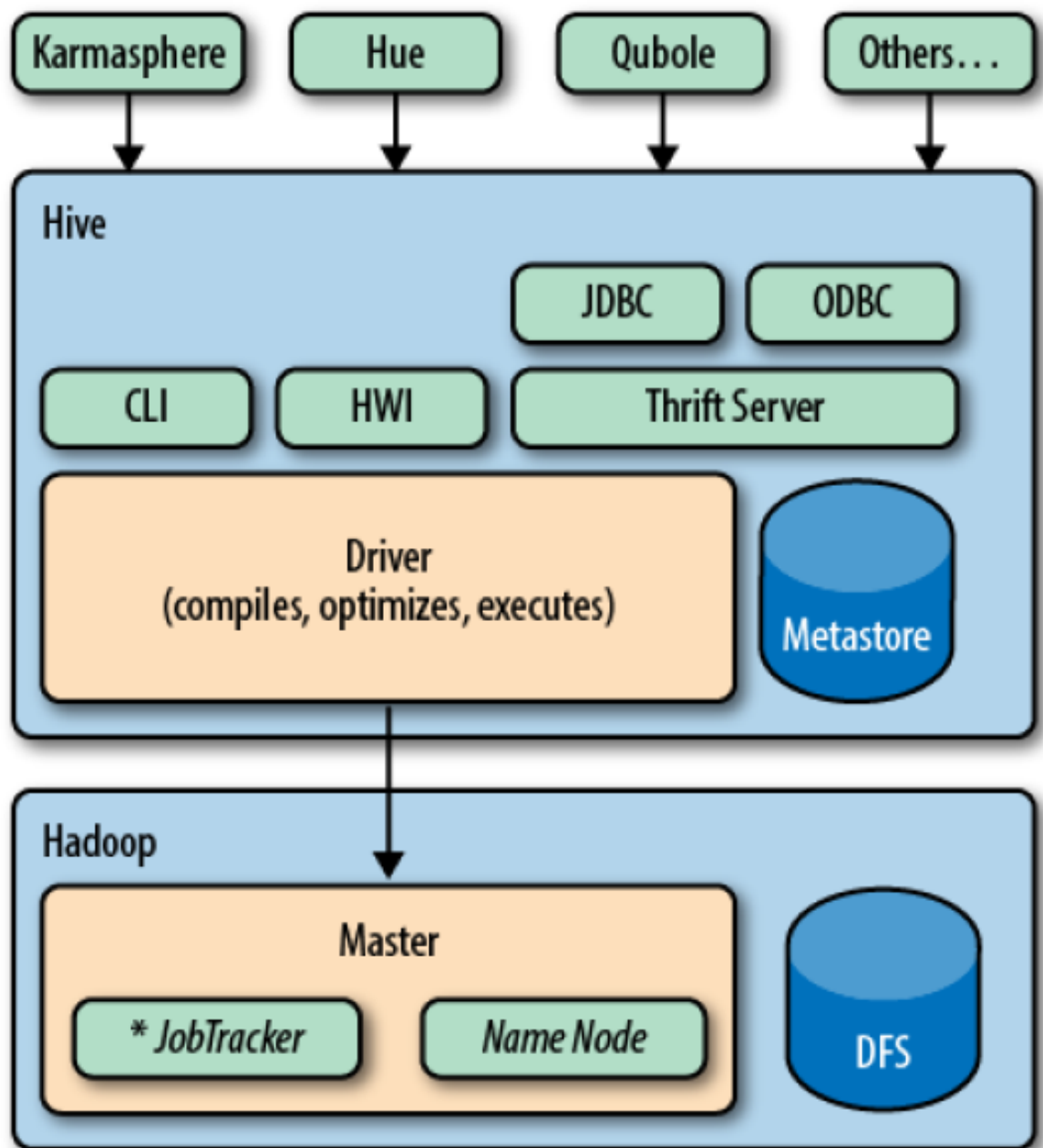


Рисунок 1.3 - Архітектура Apache Hive

Типи таблиць в Hive:

- External Tables (Зовнішні таблиці) - тип таблиць використовується, коли дані використовуються спільно з іншими інструментами (наприклад, Pig, MapReduce або Spark) або коли дані знаходяться за межами стандартної директорії Hive.
- Managed Tables (Керовані таблиці): Hive повністю керує життєвим циклом даних та зберігає їх у визначеній директорії складу. За замовчуванням дані зберігаються у підкаталозі директорії сховища (warehouse directory), яка визначається параметром конфігурації `hive.metastore.warehouse.dir` (стандартно `/user/hive/warehouse`).

Оптимізація зберігання даних: Partitioning та Bucketing

- Partitioning (Секціонування): розподіл даних на підпапки на основі значення певної колонки (наприклад, `year=2024/month=01`). Це дозволяє зчитувати лише потрібні дані, прискорюючи запити.
- Bucketing (Кластеризація/Бакетинг): хешування значень колонки в фіксовану кількість файлів (бакетів). Це покращує продуктивність операцій з'єднання (JOIN) та вибірки (SAMPLING).

Формати зберігання даних у екосистемі Hadoop (Avro, Parquet, ORC)

Вибір правильного формату файлів є одним із ключових рішень при проектуванні архітектури Big Data, оскільки він безпосередньо впливає на продуктивність виконання запитів, обсяг використаного дискового простору та швидкість передачі даних мережею. У той час як текстові формати, такі як CSV або JSON, є зручними для читання людиною та простими у використанні, вони є вкрай неефективними для обробки великих масивів даних через відсутність вбудованої схеми, неможливість паралельної обробки без повного сканування файлу та низький коефіцієнт стиснення. У сучасних екосистемах Hadoop та Spark стандартом де-факто стало використання спеціалізованих бінарних форматів, які оптимізовані для різних патернів навантаження. Серед них найбільшого поширення набули Apache Avro, Apache Parquet та Apache ORC. Розуміння архітектурних відмінностей між цими

форматами дозволяє інженерам будувати ефективні сховища даних, балансуючи між швидкістю запису та швидкістю читання.

Фундаментальна відмінність між цими форматами полягає у способі фізичної організації даних на диску, що поділяє їх на рядкові (row-oriented) та колонкові (column-oriented). Apache Avro є класичним представником рядкового формату зберігання даних. У цьому форматі всі поля одного запису зберігаються фізично поруч, блок за блоком. Така структура є ідеальною для патернів доступу, що передбачають інтенсивний запис даних або зчитування всіх полів об'єкта одночасно. Головною особливістю Avro є відокремлення схеми даних від самих даних. Схема записується у форматі JSON у заголовок файлу, тоді як дані зберігаються у компактному бінарному вигляді. Такий підхід забезпечує виняткову підтримку еволюції схеми (Schema Evolution), дозволяючи додавати, видаляти або змінювати поля без необхідності повного перезапису історичних даних. Завдяки цим властивостям Avro найчастіше використовується як формат для транспортного рівня, наприклад, у системах потокової передачі даних Apache Kafka, де критично важливим є швидкість серіалізації та надійність передачі повідомлень.

На противагу Avro, формат Apache Parquet реалізує колонковий підхід до зберігання, який був натхненний системою Dremel від Google. У файлах Parquet дані групуються не за рядками, а за колонками: спочатку зберігаються всі значення першої колонки, потім другої і так далі. Така архітектура надає колосальні переваги для аналітичних навантажень (OLAP), де запити часто оперують лише невеликою підмножиною атрибутів таблиці (наприклад, обчислення середньої ціни товару без зчитування інформації про покупця). Зчитуючи лише необхідні колонки, система значно зменшує навантаження на підсистему вводу-виводу (I/O). Крім того, зберігання однотипних даних поруч дозволяє використовувати високоефективні алгоритми стиснення, такі як Run-Length Encoding (RLE) або Dictionary Encoding, що часто зменшує обсяг файлів у декілька разів порівняно з рядковими форматами. Parquet також зберігає метадані зі статистикою (мінімум, максимум, кількість нулів) для кожної групи рядків, що дозволяє рушіям обробки, таким як Apache Spark, пропускати цілі блоки даних, які не задовольняють умовам фільтрації запиту (механізм Predicate Pushdown).

Формат Apache ORC (Optimized Row Columnar) також є колонковим, проте він був розроблений специфічно для екосистеми Apache Hive з метою подолання обмежень інших форматів. ORC поділяє файл на смуги (stripes), кожна з яких містить власні індекси та дані, організовані за колонками. Особливістю ORC є глибока інтеграція з Hive, що дозволяє досягти найвищої продуктивності при виконанні SQL-запитів саме в цьому середовищі. Він підтримує ACID-транзакції в Hive (оновлення та видалення рядків), має вбудовані легковагові індекси (Bloom filters) для швидкого пошуку та забезпечує високий ступінь стиснення даних. Хоча Parquet є більш універсальним і краще підтримується у Spark, ORC залишається найкращим вибором для побудови сховищ даних, де основним інструментом доступу є HiveQL.

Вибір конкретного формату залежить від характеру задачі: якщо пріоритетом є швидкість запису та гнучкість схеми, обирають Avro; якщо ж система орієнтована на складну аналітику та агрегацію великих обсягів даних — Parquet або ORC стануть оптимальним рішенням. Для наочності ключові характеристики цих форматів зведено у порівняльну таблицю нижче.

Таблиця 1.1 – Порівняльна характеристика форматів Avro, Parquet та ORC

Характеристика	Apache Avro	Apache Parquet	Apache ORC
Тип зберігання	Рядковий (Row-based)	Колонковий (Columnar)	Колонковий (Columnar)
Основна сфера застосування	Транзакційні системи (OLTP), потокова передача даних (Kafka), ETL (staging area).	Аналітичні системи (OLAP), Data Science, загальне використання в Spark.	Аналітичні системи (OLAP) на базі Hive, Data Warehousing.
Ефективність стиснення	Середня. Добре стискається, але поступається колонковим форматам.	Висока. Однотипні дані в колонках дозволяють використовувати ефективні кодеки (Snappy, GZIP).	Дуже висока. Часто забезпечує найменший розмір файлу в екосистемі Hive.
Підтримка еволюції схеми	Відмінна. Найкраща підтримка додавання/зміни полів.	Обмежена. Можливе додавання колонок, але це ресурсоємна операція.	Обмежена. Підтримується, але менш гнучка, ніж у Avro.

Продовження таблиці 1.1

Характеристика	Apache Avro	Apache Parquet	Apache ORC
Швидкість операцій	Висока швидкість запису (Write-heavy).	Висока швидкість читання (Read-heavy).	Висока швидкість читання, оптимізована для Hive.
Інтеграція з інструментами	Широка підтримка в Kafka, Hadoop, Spark.	Нативний формат для Apache Spark, Impala, Arrow.	Нативний формат для Apache Hive, Presto.
Особливості	Схема зберігається в JSON окремо від бінарних даних.	Підтримує складні вкладені структури даних. Ефективний Predicate Pushdown.	Підтримує ACID транзакції в Hive, має вбудовані індекси.

ПРАКТИЧНЕ ЗАВДАННЯ

Для виконання завдання використовуйте хмарні або розгорнуті локально програмні засоби Apache Hadoop версії 3+ та Apache Hive версії 3+.

Вхідні дані: Для виконання роботи рекомендується використовувати відкриті дані про юридичних осіб (ЄДРПОУ) та фізичних осіб-підприємців (ФОП), або інші великі CSV-датасети. Основне джерело: Дані Єдиного державного реєстру юридичних осіб, фізичних осіб-підприємців та громадських формувань (data.gov.ua) або підготовлені файли з лабораторного архіву . Альтернатива: Lending Club Loan Data або згенеровані синтетичні дані (> 1 млн рядків).

Завдання 1. Підготовка та завантаження даних (Python & HDFS)

Завантаження даних. Напишіть скрипт на Python для завантаження CSV файлів (UO.csv, FOP.csv) з відкритого джерела або Google Drive.

Робота з HDFS CLI:

- Створіть директорії для даних у HDFS: /tables_data/UO та /tables_data/FOP
- Завантажте отримані CSV файли у відповідні директорії HDFS .
- Перевірте наявність файлів командою `hdfs dfs -ls`.

Приклад команди:

```
hdfs dfs -mkdir -p /tables_data/UO
hdfs dfs -put local_UO.csv /tables_data/UO/
```

Завдання 2. Створення таблиць в Apache Hive

Створення зовнішніх таблиць: Створіть зовнішні таблиці `uo_table` та `for_table`, що вказують на завантажені дані. Використовуйте `OpenCSVSerde` для коректної обробки формату CSV (розділювачі, лапки).

- Схема UO: name, edrpou, address, boss, kved, stan.
- Схема FOP: fio, address, kved, stan.

Приклад SQL-запиту створення таблиці:

```
CREATE EXTERNAL TABLE IF NOT EXISTS uo_table (  
    name STRING,  
    edrpou STRING,  
    address STRING,  
    boss STRING,  
    founders STRING,  
    kved STRING,  
    stan STRING  
)  
ROW FORMAT SERDE 'org.apache.hadoop.hive.serde2.OpenCSVSerde'  
WITH SERDEPROPERTIES (  
    "separatorChar" = ",",  
    "quoteChar"     = "\""   
)  
STORED AS TEXTFILE  
LOCATION '/tables_data/UO';
```

Перевірте коректність створення таблиці, виконавши запит:

```
SELECT * FROM uo_table LIMIT 10;
```

Завдання 3. Виконання аналітичних запитів (HiveQL)

1. Агрегація даних. Підрахуйте загальну кількість записів у кожній таблиці (`count(*)`). Порівняйте час виконання з традиційними СКБД (MySQL/PostgreSQL).
2. Аналітика даних. Виведіть список юридичних осіб, додавши нумерацію рядків у межах кожної адреси (використовуючи віконну функцію `row_number() over (partition by (поле адреси) order by (код юридичної особи))`). Це дозволяє знайти адреси масової реєстрації.
3. З'єднання (Join): Виконайте JOIN таблиць `uo_table` та `for_table` за адресою реєстрації, щоб знайти перетини адрес реєстрації.

Завдання 4. Оптимізація зберігання даних (Partitioning & Bucketing)

1. Реалізація механізму партиціювання (Partitioning).

- Розробіть DDL-скрипт для створення нової керованої таблиці (Managed Table) з назвою `lending_club_part` (або `uo_part` відповідно до варіанту), застосувавши горизонтальне партиціювання за категоріальним атрибутом (наприклад, `verification_status` для фінансових даних або `stan` для реєстру юридичних осіб).
- Виконайте наповнення створеної таблиці даними з основної (зовнішньої) таблиці, використовуючи конструкцію `INSERT INTO ... SELECT`.
- Проведіть аналіз фізичної структури збереження даних у розподіленій файловій системі HDFS (команда `hdfs dfs -ls`), щоб підтвердити створення відповідних піддиректорій для кожного ключа партиціювання.

2. Застосування бакетингу (Bucketing).

- Виконайте проектування таблиці `lending_club_bucket` із використанням механізму бакетингу (хешування), розподіливши дані на 10 сегментів (buckets) за полем унікального ідентифікатора (`id` або `edrpou`).
- Перед завантаженням даних активуйте примусове бакетування (`SET hive.enforce.bucketing = true;`)[3].
- Здійсніть завантаження даних та проведіть порівняльний аналіз продуктивності: виміряйте латентність виконання точкового запиту на вибірку (`SELECT * FROM ... WHERE id = ...`) для таблиці з бакетами та аналогічного запиту до стандартної таблиці. Обґрунтуйте отримані результати.

Завдання 5. Виконання задачі WordCount за допомогою вбудованої бібліотеки

1. Підготовка вхідних даних у HDFS. Створіть директорію для вхідних даних у розподіленій файловій системі. Завантажте текстовий файл для аналізу (наприклад, книгу або набір логів).

2. Запуск MapReduce-задачі. Використовуйте утиліту `hadoop jar`. Вкажіть повний шлях до JAR-файлу прикладів версії 3+, назву програми (`wordcount`), вхідну директорію та *нову* директорію для результатів. Примітка: Якщо змінна оточення `$HADOOP_HOME` не налаштована, вкажіть повний шлях до файлу, який зазвичай знаходиться в `/usr/local/hadoop` або `/opt/hadoop`. Використайте різну кількість редюсерів: використайте прапорець `-D mapreduce.job.reduces=2` (або більше) перед назвою `jar`-файлу.

3. Моніторинг та аналіз результатів. Після завершення `job`-у перевірте вміст вихідної директорії. Результати будуть збережені у файлі з назвою `part-r-00000`, `part-r-00001`, ... Порівняйте розподіл слів між цими файлами.

ВИМОГИ ДО ЗВІТУ

Звіт до комп'ютерного практикуму виконується у Microsoft Word/PDF та повинен містити наступні розділи :

1. Титульний аркуш та відомості про виконавця.
2. Опис задачі та варіанту: джерело та структура вхідних даних.
3. Хід роботи:
 - Екранні форми виконання команд HDFS (створення папок, завантаження).
 - DDL-скрипти створення таблиць Hive (External, Partitioned, Bucketed).
 - Тексти SQL-запитів та час їх виконання (екранні форми результатів).
 - Екранні форми результату виконання MapReduce задачі (Hadoop Web UI або `cat` вихідного файлу).
4. Аналіз результатів порівняння часу виконання запитів на звичайних таблицях та оптимізованих (партиціонованих та з використанням бакетів) .
5. Аналіз порівняння розподіл слів між цими файлами з підрахованими словами `part-r-00000`, `part-r-00001`, ... в залежності від різної кількості редюсерів.
6. Висновки.

Додатково до звіту надається посилання на файл ноутбука (.ipynb) або архів з кодом.

КОНТРОЛЬНІ ПИТАННЯ

- У чому різниця між NameNode та DataNode? Що станеться, якщо вийде з ладу NameNode?
- Поясніть відмінність між External Table та Managed Table у Hive. В яких випадках краще використовувати External?
- Як працює механізм реплікації в HDFS? Для чого він потрібен?
- Що таке Hive Metastore і де він зазвичай зберігається?
- У чому перевага використання формату Parquet або ORC порівняно з TextFile (CSV)?
- Поясніть різницю між Partitioning та Bucketing. Чи можна використовувати їх одночасно?
- Яку роль виконує YARN у кластері Hadoop?